

KIXIMA

Documentação Técnica

Linguagem & Ambiente · Backend · Frontend · Schema / SQL

Versão 1.0

Setembro de 2026

Índice

- 1. Introdução
- 2. Linguagem e Ambiente de Execução
- 3. Arquitetura Geral
- 4. Backend
 - 4.1 Stack e Dependências
 - 4.2 Estrutura de Diretórios
 - 4.3 Ciclo de Vida do Pedido HTTP
 - 4.4 Autenticação e Autorização
 - 4.5 Acesso a Dados (Prisma)
 - 4.6 Tempo Real (Socket.IO)
 - 4.7 Testes
 - 4.8 Decisões de Arquitetura Notáveis
- 5. Frontend
 - 5.1 Stack e Dependências
 - 5.2 Estrutura de Diretórios
 - 5.3 Routing
 - 5.4 Estado e Acesso a Dados
 - 5.5 Internacionalização
 - 5.6 Estilo e Design
 - 5.7 Testes
- 6. Base de Dados — Schema / SQL
 - 6.1 Configuração da Fonte de Dados
 - 6.2 Convenções Gerais
 - 6.3 Modelos por Domínio
 - 6.4 Enumerações
 - 6.5 Migrações
- 7. Segurança — Resumo Transversal
- 8. Anexo: Inventário de Dependências

1. Introdução

A KIXIMA é um e-marketplace fechado para o setor petrolífero e de gás em Angola, que liga empresas compradoras a empresas fornecedoras com um modelo de "pagamento garantido": o comprador paga para dentro da plataforma, e a KIXIMA só liberta o dinheiro ao fornecedor depois de o comprador confirmar a receção conforme do que encomendou.

Este documento descreve a arquitetura técnica da plataforma tal como implementada hoje: a linguagem e o ambiente de execução, a organização do backend e do frontend, e o esquema completo da base de dados. Destina-se a quem vai manter, auditar ou integrar-se com o sistema — não repete o que já está coberto pelo "Manual do Utilizador KIXIMA" (funcionalidades do ponto de vista de quem usa a plataforma) nem pelo "AMBIENTE_TESTE.md"/"DEPLOY.md" (como publicar e operar um ambiente).

1.1 Visão de alto nível

- **Modelo de aplicação:** SPA em React consumindo uma API REST em Node.js/Express, com um canal de tempo real via Socket.IO para chat e notificações.
- **Base de dados:** um único PostgreSQL (Supabase em produção), acedido exclusivamente através do Prisma ORM — nunca SQL solto na aplicação.
- **Implantação:** imagem Docker única que compila o frontend e serve tanto a API (/api/*) como os ficheiros estáticos do SPA a partir da mesma origem — sem CORS nem proxy em produção.
- **Multi-tenant:** cada empresa (Cliente ou Fornecedor) é um tenant isolado por companyId; o isolamento é reforçado a nível de middleware (requiresSameCompany), não apenas de consulta.

2. Linguagem e Ambiente de Execução

Componente	Linguagem	Detalhe
Backend	JavaScript (Node.js)	CommonJS puro — <code>require()/module.exports</code> em 100% do código; nenhum ficheiro usa <code>import/export</code> ES Modules.
Frontend	JavaScript (JSX)	Sem TypeScript — 140 ficheiros <code>.jsx</code> + 23 <code>.js</code> , 0 ficheiros <code>.ts/.tsx</code> . Módulos ES (<code>"type": "module"</code> no <code>package.json</code>).
Base de dados	SQL (PostgreSQL)	Acesso via Prisma Client — o SQL real é gerado pelo Prisma a partir do <code>schema.prisma</code> , nunca escrito à mão na aplicação.

2.1 Versões de runtime

- Node.js **20** na imagem Docker de build/execução (`node:20-alpine`); a integração contínua (CI) corre os testes contra Node **22**.
- Sem campo `engines` fixado no `package.json` do backend — a versão é imposta pela imagem Docker, não pelo npm.
- Gestor de pacotes: npm (`npm ci` nos Dockerfiles, com `package-lock.json` comitado).

3. Arquitetura Geral

A plataforma segue uma arquitetura em três camadas clássica, com uma separação estrita entre apresentação, lógica de negócio e persistência:

- **Cliente** — SPA React, servido estaticamente pelo próprio backend em produção (mesma origem).
- **API** — Express, organizada em rotas → (controladores opcionais) → serviços → Prisma. A maioria das rotas fala diretamente com os serviços; os controladores só existem para um subconjunto de domínios.
- **Persistência** — um único PostgreSQL, com o Prisma Client como única porta de entrada/saída de dados.

3.1 Comunicação em tempo real

Um canal Socket.IO paralelo à API REST cobre o que precisa de latência baixa e não se presta a pedido/resposta: chat de suporte, chat comercial e notificações. Reutiliza a mesma lista de origens CORS que o Express, para as duas configurações nunca divergirem.

3.2 Isolamento multi-empresa (multi-tenant)

Cada empresa é um tenant lógico dentro da mesma base de dados (não há uma base por cliente). O isolamento é garantido em dois pontos: o token de sessão transporta o `companyId` do utilizador, e o middleware `requireSameCompany()` verifica-o explicitamente antes de qualquer leitura/escrita sensível — nunca se confia apenas no filtro da própria consulta.

4. Backend

4.1 Stack e Dependências

Dependências de produção

Categoria	Pacote	Versão
Framework web	express	^4.19.2
Framework web	express-async-errors	^3.1.1
Framework web	express-rate-limit	^8.6.2
Segurança / cabeçalhos	helmet	^7.1.0
Segurança / CORS	cors	^2.8.5
Segurança	cookie / cookie-parser	^0.7.2 / ^1.4.7
Autenticação	jsonwebtoken	^9.0.2
Autenticação	bcryptjs	^2.4.3
ORM	@prisma/client	^5.20.0
Validação	zod	^3.23.8
Tempo real	socket.io	^4.8.3
Email	nodemailer	^9.0.4
Armazenamento / S3	@aws-sdk/client-s3	^3.1091.0
Upload	multer	^1.4.5-lts.1
Folhas de cálculo	xlsx	^0.18.5
Mensageria (ERP)	amqplib	^0.10.9
Agendamento	node-cron	^3.0.3
Observabilidade	@sentry/node	^10.69.0
Logging HTTP	morgan	^1.10.0

Dependências de desenvolvimento

Categoria	Pacote	Versão
CLI do ORM	prisma	^5.20.0
Testes	jest	^29.7.0
Testes	supertest	^7.0.0

Sem ESLint/Prettier configurado neste pacote. overrides.uuid força uma versão de uma dependência transitiva.

4.2 Estrutura de Diretórios (backend/src/)

Pasta	Ficheiros	Responsabilidade
config/	7	Singletons transversais: ambiente, Prisma Client, logger, listas CORS/CSP, Sentry, configuração de upload.
routes/	30	Um router Express por domínio de negócio; a maioria fala diretamente com os serviços.

Pasta	Ficheiros	Responsabilidade
controllers/	8	Camada HTTP fina para um subconjunto de rotas (8 de 30 ficheiros de rota passam por aqui).
services/	59	O grosso da lógica de negócio — um módulo por concern (faturação, subscrições, risco, notificações, tempo real, armazenamento, etc.).
middleware/	4	auth.js (JWT), rbac.js (papéis/permisões), errorHandler.js, rateLimit.js.
jobs/	5	Tarefas agendadas (cron): cópias de segurança, lembretes de MFA, expiração de apólices/retenção/subscrição.
utils/	10	Auxiliares sem estado: classes de erro, paginação, política de senhas, TOTP, esquemas, cookie de sessão, validação de assinatura de ficheiro.
i18n/	1	Strings de email traduzidas.
workers/	1	Análise isolada de folhas Excel (importação de catálogo).

Padrão de arquitetura: **rotas** → (**controladores opcionais**) → **serviços** → **Prisma** — mais orientado a serviços do que a controladores.

4.3 Ciclo de Vida do Pedido HTTP

Cadeia de middleware em `backend/src/app.js`, pela ordem em que é montada:

- `express-async-errors` — carregado antes de tudo, para qualquer erro assíncrono chegar sempre ao tratador central.
- `Sentry` (instrumentado o mais cedo possível).
- `helmet` com CSP customizado.
- `cors` com lista de origens.
- `cookie-parser` e `express.json()` (com captura do corpo em bruto, para verificar assinaturas HMAC de webhooks de pagamento).
- Ficheiros estáticos de upload + fallback 404 em JSON.
- `morgan` (omitido em ambiente de teste).
- Sondas de disponibilidade `/health` e `/ready`.
- Limitador de pedidos geral em `/api/`, mais limitadores apertados por rota sensível (login, 2FA, recuperação de senha, registo/convite).
- ~30 routers de domínio montados sob `/api/*`.
- Serviço estático do SPA (produção) com rota universal de fallback.
- `notFoundHandler` e `errorHandler` finais.

4.4 Autenticação e Autorização

Autenticação (`middleware/auth.js`)

- O token é lido primeiro de um **cookie de sessão `httpOnly`**; só na ausência deste se aceita um cabeçalho `Authorization: Bearer` — de propósito, para um script injetado no browser nunca poder substituir a sessão real por um cabeçalho forjado.
- Assinatura verificada só com HS256.
- **Revogação do lado do servidor:** compara-se o `tv` (versão do token) do payload com o `tokenVersion` do utilizador na base — permite invalidar todas as sessões antigas ao mudar a senha, terminar sessão em todo o lado, ou bloquear uma conta.

- **MFA em transição:** antes do prazo definido para 2FA obrigatória, a sessão funciona normalmente com um aviso; depois do prazo, uma conta sem 2FA ativa fica com sessão restrita — só pode aceder às rotas necessárias para ativar a 2FA, nunca fica trancada de vez.

Autorização por papel e por área (middleware/rbac.js)

- Cinco perfis: COMPRADOR, COMPANY_ADMIN, FORNECEDOR, FINANCEIRO, ADMIN_SISTEMA.
- `requireRole(...)` — lista de papéis permitidos.
- `requireSameCompany()` — isolamento de tenant; o Admin do Sistema está isento.
- `requirePermission(area)` — dentro do papel Admin do Sistema, restringe por uma das 6 áreas (adminAreas); um array vazio de áreas significa Super Admin (acesso total), de propósito.
- `requireSuperAdmin()` — reservado à própria gestão de acessos (conceder áreas, bloquear contas) — nunca delegável a um admin restrito a uma área.

4.5 Acesso a Dados (Prisma)

Uma única instância partilhada de `PrismaClient` (`config/database.js`) — todo o resto do código importa a partir daqui, nunca instancia o seu próprio client. A instância é envolvida com `$extends` para aplicar um teto por omissão (configurável, 1000 linhas) a todas as consultas `findMany` sem um `take` explícito. Se o resultado atingir exatamente o teto — sinal de corte quase certo — a aplicação regista um aviso de nível `error` com o modelo e o ponto do código que fez a chamada, em vez de devolver silenciosamente um total truncado com ar de certo. Um pequeno conjunto de modelos pequenos e sensíveis à contagem exata fica isento deste teto.

4.6 Tempo Real (Socket.IO)

- Inicializado em `services/realtimeService.js`, reutilizando a mesma lista de origens CORS do Express.
- Middleware de autenticação do socket popula `socket.user` a partir do mesmo JWT.
- Sem namespaces customizados — só o namespace / por omissão, com salas.
- Cada socket entra automaticamente numa sala pessoal `user:<userId>`.
- Salas dinâmicas por pedido do cliente (`support:join`, `conversation:join`, etc.) — o servidor **revalida sempre contra a base de dados** antes de aceitar a entrada na sala, nunca confia no identificador que o cliente enviou.

4.7 Testes

- Executor: `jest ^29.7.0 + supertest ^7.0.0`, contra a app exportada (nunca um servidor a correr de verdade).
- **84** ficheiros de teste (`tests/*.test.js`), cerca de **770** casos de teste.
- Base de dados de teste provisionada uma vez por execução (`globalSetup`); `maxWorkers: 1` — os testes correm em série porque partilham a mesma base.

4.8 Decisões de Arquitetura Notáveis

Ficheiro	Princípio de desenho declarado no cabeçalho
<code>faturacaoService.js</code>	A série de numeração de faturas é por fornecedor, nunca global — cada empresa fornecedora é o seu próprio emitente fiscal.
<code>assinaturaService.js</code>	Dois caminhos de pagamento: transferência com comprovativo é sempre confirmada por humano; só os planos BASE/CORE (nunca o PRO) podem usar um canal automático.

Ficheiro	Princípio de desenho declarado no cabeçalho
riskAnalysisService.js	Deteção de negociação fora da plataforma por pontuação de vários sinais , nunca por lista de palavras proibidas — uma palavra isolada dá demasiados falsos positivos.
notificationService.js	Centraliza toda a regra "quem é avisado, por que canal" num único sítio; os fornecedores de email/in-app são intercambiáveis.
config/database.js	O teto do findMany é uma rede de segurança, não paginação — um corte silencioso é considerado pior do que um crash, por isso grita em vez de devolver um total errado.

5. Frontend

5.1 Stack e Dependências

Dependências de produção

Categoria	Pacote	Versão
Core	react / react-dom	^18.3.1
Routing	react-router-dom	^7.18.2
Tempo real	socket.io-client	^4.8.3
Observabilidade	@sentry/react	^10.69.0
App nativa (híbrida)	@capacitor/core, /android, /ios	^8.5.0
Tipografia	@fontsource-variable/inter, /sora, ibm-plex-mono	^5.2.x
Utilitário	qrcode	^1.5.4

Dependências de desenvolvimento

Categoria	Pacote	Versão
Build	vite / @vitejs/plugin-react	^5.4.6 / ^4.3.1
Testes unitários	vitest	^4.1.10
Testes unitários	@testing-library/react, jest-dom, user-event	diversas
Testes E2E	@playwright/test	^1.62.1
Acessibilidade (E2E)	@axe-core/playwright	^4.13.0
App nativa	@capacitor/cli	^8.5.0

Ausências notáveis: sem axios (o cliente HTTP é um wrapper próprio sobre fetch), sem Redux/Zustand/React Query (estado via Context + estado local), sem Tailwind/CSS-in-JS, sem biblioteca de componentes de terceiros.

5.2 Estrutura de Diretórios (frontend/src/)

Pasta	Ficheiros	Responsabilidade
pages/shared	26	Ecrãs comuns a todas as personas (login, registo, perfil, notificações, ajuda, documentos imprimíveis).
pages/adminSistema	20	Ecrãs do Admin do Sistema.
pages/comprador	18	Ecrãs do Comprador + CartContext.
pages/fornecedor	18	Ecrãs do Fornecedor.
pages/companyAdmin	13	Ecrãs do Administrador da Empresa.
pages/corporate	8	Site institucional público (home, notícias, carreiras, FAQ).
pages/financeiro	3	Ecrãs do Financeiro.
components/	25 (+3)	Primitivas de UI partilhadas (Sidebar, Navbar, Button, DataTable, Badge, widgets de chat).
i18n/	14	Motor de tradução + 12 ficheiros de dicionário (content.js...content12.js).

Pasta	Ficheiros	Responsabilidade
auth/	3	AuthContext + guarda de rotas RequireAuth.
realtime/	2	RealtimeContext (ligação Socket.IO partilhada).
api/	2	Cliente HTTP central (client.js).
data/	3	Configuração da barra lateral e taxonomia de categorias (UNSPSC).
styles/	1	global.css — 3184 linhas, única folha de estilo.

Organização por persona: as páginas ficam divididas em subpastas por papel — espelhando os 5 perfis autenticados da plataforma.

5.3 Routing

- react-router-dom v7, 118 entradas <Route> em App.jsx.
- **Code-splitting extensivo:** todas as páginas exceto o ecrã de login são carregadas via React.lazy(), dentro de um único <Suspense> de topo.
- Rotas públicas (sem sessão): login, cadastro, convites, recuperação de senha, páginas institucionais, planos.
- Bloco autenticado protegido por <RequireAuth />, que monta o RealtimeProvider + CartProvider + a moldura da aplicação.
- Acesso por papel via <RequireRole role="X" /> por bloco de persona (aceita um papel único ou uma lista, para os ecrãs partilhados por duas personas).
- **Redirecionamento por papel:** um mapa ROLE_HOME resolve cada um dos 5 papéis para o seu painel — usado tanto na página inicial (visitante → home pública; autenticado → o seu painel) como quando alguém tenta aceder a um ecrã fora do seu papel (é reencaminhado para o seu próprio painel, nunca para /login).

5.4 Estado e Acesso a Dados

Cliente API (src/api/client.js)

- Wrapper fino sobre fetch nativo — sem axios.
- **Modelo de autenticação:** um cookie httpOnly definido pelo backend é o caminho principal; o cliente nunca lê/escreve um token no localStorage para a versão Web — decisão deliberada contra XSS.
- Exceção só para a app nativa (Capacitor): um token Bearer fica em memória (nunca persistido), porque cookies entre origens podem não sobreviver dentro da WebView.
- As mensagens de erro do servidor chegam já traduzidas — a tradução acontece no próprio limite do cliente antes de devolver o erro a quem chamou.

Estado global — React Context (sem Redux/Zustand)

Context	Gere
AuthContext	Utilizador atual, estado de carregamento, fluxo de login/2FA, logout, atualização de perfil.
RealtimeContext	Ligação Socket.IO única e partilhada por toda a app; entrar/sair de salas de suporte, chat comercial e notificações.
CartContext	Cesta de compras do Comprador — montado só dentro da moldura autenticada, âmbito da persona Comprador.
I18nContext	Idioma ativo e a função de tradução t().

5.5 Internacionalização

- Três idiomas: português (pt-AO), inglês (en-GB), francês (fr-FR).
- **As próprias strings em português funcionam como chave de tradução** — `t('Ordens de Compra')` procura essa string no dicionário do idioma ativo, e cai de volta ao próprio português se não houver tradução.
- Dicionários fragmentados em 12 ficheiros (`content.js ... content12.js`), fundidos num único objeto em runtime.
- Idioma escolhido guardado em `localStorage` e espelhado no servidor (para localizar conteúdo gerado do lado do servidor, como emails).
- Auditoria de cobertura de tradução com scripts próprios (`i18n:check`, `i18n:missing`, `i18n:unused`, `i18n:hardcoded`).

5.6 Estilo e Design

CSS simples e global — um único ficheiro `global.css` (3184 linhas), sem CSS Modules, sem Tailwind, sem biblioteca de componentes de terceiros. As primitivas de UI (Button, Badge, DataTable, etc.) são construídas à mão e estilizadas por classes contra esse ficheiro, com um sistema de tokens em variáveis CSS (cores, espaçamento). Tipografia autoalojada via pacotes `@fontsource` (Inter, Sora, IBM Plex Mono) em vez de carregada de uma CDN.

5.7 Testes

- `vitest`, ambiente `jsdom` — 21 ficheiros de teste, cobrindo o módulo de domínio, o cliente API, os Contexts, os dicionários de `i18n` e várias páginas.
- Suite E2E separada com `Playwright` (incluindo verificações de acessibilidade via `@axe-core/playwright`), fora da execução do Vitest.

6. Base de Dados — Schema / SQL

6.1 Configuração da Fonte de Dados

`provider = "postgresql"`. Duas connection strings distintas, por desenho: `url` (env `DATABASE_URL`) aponta ao pooler de **transação** do Supabase (porta 6543, `pgbouncer=true`) — usado pela aplicação em runtime; `directUrl` (env `DIRECT_URL`) aponta ao pooler de **sessão** (porta 5432) — usado só para migrações/DDL, que precisam de uma ligação direta e persistente para operações como `CREATE DATABASE/DDL` transacional.

6.2 Convenções Gerais

- `@@map`: todos os modelos mapeiam para uma tabela em `snake_case` (ex.: `PurchaseOrder` → `purchase_orders`); o mesmo a nível de campo (`companyId` → `company_id`).
- `@@index`: quase todas as chaves estrangeiras têm um índice explícito; índices compostos cobrem padrões de consulta reais (ex.: `Invoice @@index([serie, numeroNaSerie])`).
- `@@unique`: usado para chaves de negócio (`PurchaseOrder.reference`) e para unicidade de junção (`Review @@unique([productId, userId])`).
- Valores monetários são sempre `Decimal` com precisão explícita (`@db.Decimal(14, 2)` na maioria dos casos) — nunca `Float`.
- Relações nomeadas (`@relation("Nome", ...)`) sempre que um modelo se liga duas vezes ao mesmo alvo (ex.: `Company` ↔ `PurchaseOrder` como comprador e como fornecedor).
- Padrão de "porta fechada por omissão": vários modelos guardam campos nulos até uma funcionalidade ser explicitamente ativada por uma empresa (ex.: a cadeia de certificação fiscal AGT só começa a preencher-se depois da empresa aderir).

6.3 Modelos por Domínio

43 modelos, agrupados por domínio de negócio.

Identidade & Acesso

Modelo	Propósito	Campos-chave
Company	Empresa-tenant (cliente ou fornecedora)	<code>taxId</code> , <code>type</code> , <code>status</code> , <code>verified</code> , <code>plan</code> , <code>searchRank</code> , <code>serieFiscal</code>
User	Utilizador/persona ligado a uma empresa (ou nenhuma, para Admin do Sistema)	<code>role</code> , <code>adminAreas</code> , <code>mfaMethod</code> , <code>tokenVersion</code> , <code>falhasSeguidas/bloqueadoAte</code>
EmployeeInvite	Convite emitido por um Company Admin	<code>role</code> , <code>adminAreas</code> , <code>token</code> , <code>status</code> , <code>expiresAt</code>
ApiKey	Credencial de API de catálogo por empresa (plano PRO)	<code>nome</code> , <code>prefixo</code> , <code>hash</code> , <code>revogadaEm</code>
CompanyDocument	Documento de credenciamento no cadastro	<code>type</code> , <code>fileUrl</code>
CompanyErpConfig	Configuração de integração ERP por empresa (credenciais cifradas)	<code>erp</code> , <code>configEnc</code> , <code>lastTestAt/Ok</code>
CompanyErpConfig Audit	Auditoria de alterações à configuração ERP	<code>action</code> , <code>fromErp</code> , <code>toErp</code> , <code>actorName</code>
SavedSearch	Pesquisa de marketplace guardada pelo Comprador	<code>label</code> , <code>query</code>

Subscrições / Cobrança

Modelo	Propósito	Campos-chave
PlanoCobranca	Pedido de mudança de plano e o respetivo pagamento (manual ou automático)	referencia, planoNovo, valorUsd, status, canal, referenciaExterna

Catálogo

Modelo	Propósito	Campos-chave
Product	Item de catálogo (produto ou serviço)	sku, category, unspscCode, unitPrice, stockQuantity, kind, searchText
ProductImage	Imagem do produto	url, isPrimary, sortOrder
ProductDocument	Documento técnico anexado ao produto	type, fileUrl
Kit	Conjunto de produtos vendido em pacote	name, active
KitItem	Linha de um Kit	quantity
StockMovement	Movimento de stock (entrada/saída)	type, quantity, note
Favorite	Produto favorito de um utilizador	(par único user+product)
Review	Avaliação de um produto por um comprador	rating, comment

Comercial (Cotações, Ordens, Contratos)

Modelo	Propósito	Campos-chave
QuoteRequest	Pedido de cotação (RFQ) do comprador a um fornecedor	status, responsePrice, responseLeadDays
QuoteItem	Linha de um pedido de cotação	quantity
PurchaseOrder	Ordem de compra — entidade central do fluxo	reference, status, totalAmount, isCallOff, receptionStatus
PurchaseOrderItem	Linha de uma ordem de compra	quantity, unitPrice, lineTotal
Contract	Contrato-quadro entre cliente e fornecedor	reference, billingPeriodicity, paymentTermDays, status
ReferenceCounter	Contador atómico de referências sequenciais (PO-, SD-...)	key, value

Faturação & Pagamentos

Modelo	Propósito	Campos-chave
Invoice	Fatura gerada automaticamente ao aceitar uma PO ou consolidar call-offs	reference, amount, status, serie/numeroNaSerie/hash (cadeia AGT)
InvoiceLine	Linha fiscal da fatura (discriminação de imposto exigida pela AGT)	lineNumber, unitPrice, netAmount, ivaAmount, ivaTaxCode
CreditNote	Documento fiscal de correção contra uma fatura	reference, motivo, amount, série/hash próprios
SerieFaturacao	Série de numeração declarada à AGT (por empresa/ano)	codigo, ano, ultimoNumero, ultimoHash, ativa
LinhaExtrato	Linha de extrato bancário importada para conciliação	idNoBanco, montante, estado

Modelo	Propósito	Campos-chave
Payment	Registo de pagamento contra uma fatura (também o Recibo fiscal "RC")	amount, status, canal, série/hash próprios
PlatformFee	Comissão da KIXIMA sobre uma fatura	poCount, amount, basis, fxRate, status
BudgetLimit	Teto de gasto mensal opcional por empresa	periodMonthly, currency

Apólices (Seguro)

Modelo	Propósito	Campos-chave
SupplierToKixima Policy	Apólice do fornecedor para a KIXIMA, no onboarding	policyNumber, coverageAmount, status
KiximaToClient Policy	Apólice da KIXIMA para o cliente, após due diligence	policyNumber, coverageAmount, issuedById

Comunicação

Modelo	Propósito	Campos-chave
Notification	Evento de notificação in-app/email	type, channel, readAt, relatedEntityType/Id (polimórfico)
SupportCategoryImage	Imagem de uma categoria de ajuda	key, imageUrl
SupportTicket	Pedido de apoio ao Suporte	reference, category, status, assignedToId
SupportMessage	Mensagem dentro de um pedido de apoio	authorRole, body, readAt
Conversation	Conversa comercial entre comprador e fornecedor	contextType/contextId (polimórfico), status
ConversationMessage	Mensagem dentro de uma conversa comercial	senderCompanyId, body, readAt

Confiança e Segurança

Modelo	Propósito	Campos-chave
RiskAlert	Alerta de suspeita de negociação fora da plataforma	level, signals (Json), status, decision

Auditoria

Modelo	Propósito	Campos-chave
AuditLog	Trilho de auditoria append-only, ator desnormalizado	action, entityType/Id, actorName/Role, ip, detail (Json)

Outros

Modelo	Propósito	Campos-chave
SupplierDevRequest	Candidatura pública ao programa Supplier Development	reference, track, accessFeeUsd, status
Feedback	Avaliação/testemunho público da plataforma	categoria, targetLabel, rating, approved

6.4 Enumerações

31 enumerações no total. As mais extensas:

Enum	Valores
PoStatus	AGUARDANDO_APROVACAO, APROVADA, REJEITADA, ACEITE_FORNECEDOR, RECUSADA_FORNECEDOR, AGUARDANDO_PAGAMENTO, PAGA, EM_EXECUCAO, ENTREGUE, RECEBIDA_CONFORME, RECEBIDA_COM_DIVERGENCIA, CONCLUIDA
NotificationType	24 valores — um por evento de negócio notificável (PO_APROVADA, FATURA_GERADA, PAGAMENTO_PROCESSADO, ALERTA_SEGURANCA, etc.)
CanalCobranca	TRANSFERENCIA_MANUAL, EMIS_MULTICAIXA, PAYPAY, BAI, BFA, STANDARD_BANK_ANGOLA
CompanyStatus	PENDENTE, APROVADA, REJEITADA, SUSPENSA
CobrancaStatus	PENDENTE, COMPROVATIVO_ENVIADO, CONFIRMADA, CANCELADA
PersonaRole	COMPRADOR, COMPANY_ADMIN, FORNECEDOR, FINANCEIRO, ADMIN_SISTEMA
InvoiceStatus	PENDENTE, PAGA, VENCIDA, CANCELADA
RiskLevel / RiskAlertStatus	LOW/MEDIUM/HIGH/CRITICAL — ABERTO/EM_ANALISE/FALSO_POSITIVO/RESOLVIDO
SupportStatus	ABERTO, EM_ANDAMENTO, AGUARDANDO_RESPOSTA, RESOLVIDO, FECHADO
CompanyErpSystem	MANUAL, PRIMAVERA, SAP_S4HANA, ORACLE_ERP_CLOUD, SAP_ARIBA

Restantes 21 enums (menores, 2-8 valores cada): CompanyType, CompanySize, CompanyPlan, DocumentType, PlatformFeeStatus, InviteStatus, ProductKind, QuoteStatus, StockMovementType, ProductDocType, PaymentStatus, CanalPagamento, PolicyStatus, ContractStatus, BillingPeriodicity, NotificationChannel, ConversationStatus, SupplierDevTrack, SupplierDevStatus, FeedbackCategoria.

6.5 Migrações

- 41 migrações versionadas em backend/prisma/migrations/, aplicadas com prisma migrate deploy — nunca db push em produção.
- **Migrações são bloqueadas depois de publicadas:** uma vez aplicada em produção, o Prisma trata uma migração como já registada e nunca a corre outra vez — editar o SQL de uma migração já publicada deixaria a produção dessincronizada em silêncio.
- O script migration-lock.js guarda um checksum SHA-256 de cada migração publicada; um teste dedicado recalcula os checksums a partir do disco e falha se algum mudar. Uma alteração de esquema é sempre uma **migração nova**, nunca a edição de uma antiga.

7. Segurança — Resumo Transversal

- **Sessão:** JWT assinado (HS256) num cookie httpOnly — nunca em localStorage no lado Web; revogação por versão de token permite invalidar sessões antigas sem esperar a expiração natural.
- **2FA/MFA:** obrigatória por papel (Company Admin e Admin do Sistema), por email ou aplicação TOTP; imposição faseada com sessão restrita (nunca bloqueio total) até ser ativada.
- **Controlo de acesso:** RBAC por papel + sub-permissões por área para o Admin do Sistema (Cadastro, Financeiro, Faturação/AGT, Apólices, Suporte, Operação); isolamento de tenant reforçado por middleware, não apenas por consulta.
- **Senhas:** hash com bcrypt; bloqueio progressivo de conta por tentativas falhadas (nunca por IP, para não penalizar um escritório inteiro atrás do mesmo endereço).
- **Limitação de pedidos:** geral por IP/utilizador na API inteira, mais apertada em endpoints sensíveis (login, 2FA, registo, chat).
- **Cabeçalhos HTTP:** Helmet com CSP customizado; CORS restrito a uma lista de origens explícita.
- **Auditoria:** trilho append-only com ator desnormalizado — sobrevive à remoção da conta que praticou a ação; escrito dentro da mesma transação nas operações que envolvem dinheiro.
- **Integridade fiscal:** faturas, notas de crédito e recibos formam uma cadeia de hash SHA-256 encadeada por série/empresa — qualquer alteração ou buraco na numeração é detetável.
- **Webhooks de pagamento:** nunca confiam no estado que o próprio pedido afirma — o servidor reconfirma sempre a verdade junto da API do gateway antes de agir.

8. Anexo: Inventário de Dependências

Resumo consolidado de todas as dependências diretas (produção + desenvolvimento) dos dois pacotes.

backend/package.json

Pacote	Versão
express	^4.19.2
express-async-errors	^3.1.1
express-rate-limit	^8.6.2
helmet	^7.1.0
cors	^2.8.5
cookie / cookie-parser	^0.7.2 / ^1.4.7
jsonwebtoken	^9.0.2
bcryptjs	^2.4.3
@prisma/client	^5.20.0
zod	^3.23.8
socket.io	^4.8.3
nodemailer	^9.0.4
@aws-sdk/client-s3	^3.1091.0
multer	^1.4.5-lts.1
xlsx	^0.18.5
amqplib	^0.10.9
node-cron	^3.0.3
@sentry/node	^10.69.0
morgan	^1.10.0
prisma	^5.20.0
jest	^29.7.0
supertest	^7.0.0

frontend/package.json

Pacote	Versão
react / react-dom	^18.3.1
react-router-dom	^7.18.2
socket.io-client	^4.8.3
@sentry/react	^10.69.0
@capacitor/core, /android, /ios	^8.5.0
@fontsource-variable/inter, /sora, ibm-plex-mono	^5.2.x
qrcode	^1.5.4
vite / @vitejs/plugin-react	^5.4.6 / ^4.3.1

Pacote	Versão
vitest	^4.1.10
@testing-library/react, jest-dom, user-event	diversas
@playwright/test	^1.62.1
@axe-core/playwright	^4.13.0
@capacitor/cli	^8.5.0